

# LLMs versus the Halting Problem

## Characterizing Program Termination Reasoning

MDL23CS001 23CSB01 AADITH N

CSQ413 Seminar

11/09/2026

# Outline

- 1 Introduction & Background
- 2 Problem Statement & Witness Automata
- 3 Dataset & Experimental Setup
- 4 Evaluation & Main Results
- 5 Detailed Analysis & Failure Modes
- 6 Precondition Prediction
- 7 Conclusion & Future Work
- 8 References & Q&A

# Introduction & Motivation

- **The Halting Problem:** Alan Turing (1936) established that universal program termination detection is fundamental and undecidable.
- **Software Safety Risks:** Non-termination bugs trigger infinite loops, unbound CPU/memory consumption, and application freezing.
- **LLMs in Program Analysis:** Frontier Large Language Models exhibit high capability in code generation and semantic comprehension across diverse languages.
- **Research Objective:** Quantify the extent to which LLMs can predict program termination and construct machine-verifiable symbolic proofs.

# Traditional Verification vs. LLM Approach

- **Traditional Solvers (e.g., PROTON, UAutomizer):**

- Rely on complex, language-dependent multi-stage pipelines.
- Combine SMT solvers, abstract interpretation, and model checking.
- Highly sound but bounded by strict symbolic formalisms.

- **Large Language Model Approach:**

- Direct semantic reasoning over source code without explicitly compiling.
- Agnostic to programming language boundaries and syntax variations.
- Unified prompt-based interface for predicting verdicts and proofs.

# Problem Statement & Decision Outcomes

Given a C program with non-deterministic variables (`__VERIFIER_nondet_int()`), the model must evaluate semantic execution paths and output a verdict:

- **T (Terminating)**: The program halts for every valid assignment of input variables.
- **NT (Non-Terminating)**: The program diverges on at least one input path. Requires a **Witness Automaton Graph** as formal proof.
- **UNK (Unknown)**: Returned when the model cannot guarantee or determine termination behavior.

# Witness Automaton Proof Concept

- **What is a Witness Automaton?**

- A directed graph modeling infinite execution paths in non-terminating programs.
- **Nodes:** Represent program states (e.g., entry point, cycle head).
- **Edges:** Represent control-flow transitions annotated with line numbers and variable assumptions.

- **Key Advantage:**

- Does not require a fully concrete execution trace.
- Demonstrates a recurring cycle of program states that guarantees divergence.

# Witness Automaton Graph & Validation Pipeline

- **JSON Specification:** The LLM generates nodes, edges, and conditions in JSON format representing execution states.
- **GraphML Conversion:** The JSON schema is converted into a standard GraphML representation used by SV-COMP.
- **Automated Validation:** Evaluated using **UAutomizer** via symbolic analysis and SMT solvers in  $< 1$  second.
- **Validation Requirement:** Must formally prove path feasibility and cycle recurrence for infinite execution.

# Witness Automaton Cycle Diagram

## C code example:

```
1: typedef enum {false,true} bool;
2:
3: extern int __VERIFIER_nondet_int(void);
4:
5: int main() {
6:   int i;
7:   i = __VERIFIER_nondet_int();
8:
9:   while (i >= -5 && i <= 5) {
10:    if (i > 0) {
11:      i = i-1;
12:    }
13:    if (i < 0) {
14:      i = i+1;
15:    }
16:  }
17:
18:  return 0;
19: }
```

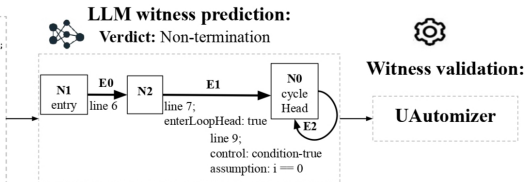


Figure: Witness Automaton Cycle Diagram



# SV-COMP 2025 Dataset Preparation

- **Benchmark Source:** SV-COMP 2025 Termination Category (2, 328 C program tasks).
- **Subcategory Distribution:**
  - **Main-ControlFlow:** Loop conditions, branching, recursion.
  - **BitVectors:** Bit-wise arithmetic operations.
  - **Main-Heap:** Dynamic memory allocations and pointers.
  - **Other:** Mixed data structures and concurrent constructs.
- **Class Balance:** Skewed towards terminating code (65.6% Terminating vs. 34.4% Non-Terminating).
- **Token Lengths:** Median length of 6K tokens (90% of tasks  $\leq$  20K tokens).

# Evaluated Models & Pipeline

- **Proprietary Reasoning Models:** GPT-5, Claude Sonnet-4.5.
- **Open-Weights Reasoning Models:** CWM (Code World Model), Qwen3-32B.
- **Baseline (Non-Reasoning):** GPT-4o.
- **Verification Tool Baselines:** PROTON (1st place), UAutomizer (2nd place), AProVE (3rd place).
- **Inference Setup:**
  - Zero-shot structured prompting targeting JSON outputs.
  - No domain-specific fine-tuning to evaluate raw reasoning potential.

# Test-Time Scaling (TTS) & SV-COMP Metric

- **Test-Time Scaling (TTS) via Consensus Voting:**

- Sample 10 out of 20 generated trajectories per instance.
- If all 10 predictions agree → output unanimous prediction.
- If any disagreement occurs → output **UNK (Unknown)** to avoid penalties.

- **SV-COMP Asymmetric Scoring Scheme:**

| Prediction Outcome                           | Score |
|----------------------------------------------|-------|
| Correct Non-Termination (NT + Valid Witness) | +2    |
| Correct Termination (T)                      | +1    |
| Unknown (UNK) / NT + Invalid Witness         | 0     |
| Incorrect Non-Termination (False Positive)   | -16   |
| Incorrect Termination (False Negative)       | -32   |

# SV-COMP Main Benchmark Results

- Reasoning models with Test-Time Scaling (TTS) reach human-engineered verifier performance levels.
- GPT-5 (Score: 3,520)** and **Claude Sonnet-4.5 (Score: 3,448)** rank 2nd and 3rd overall, trailing only **PROTON (3,685)**.
- CWM (Score: 3,129)** outperforms classical verifier AProVE (2,219) and closely trails UAutomizer (3,334).
- GPT-4o (Score: 546)** highlights the stark gap between standard LLMs and reasoning-focused models.

# Class-Wise F1-Score Performance

| Model Strategy          | F1-Score ( $T$ ) $\uparrow$ | F1-Score ( $NT$ ) $\uparrow$ |
|-------------------------|-----------------------------|------------------------------|
| GPT-5 (TTS)             | <b>0.98</b>                 | <b>0.96</b>                  |
| Claude Sonnet-4.5 (TTS) | 0.94                        | 0.89                         |
| CWM (TTS)               | 0.84                        | 0.63                         |
| Qwen3-32B (TTS)         | 0.83                        | 0.59                         |
| GPT-4o (TTS Baseline)   | 0.73                        | 0.50                         |

- Higher classification stability on Terminating tasks ( $T$ ) compared to Non-Terminating tasks ( $NT$ ).
- TTS improves overall safety score by converting ambiguous outputs into UNK, slightly reducing raw F1 metrics.

# The Classification-Proof Gap

- **Key Observation:** Models consistently recognize non-terminating loops semantically, but frequently fail to generate a machine-valid witness graph.
- **Comparison to Theorem Proving:** Similar to mathematical reasoning LLMs that deduce the correct numerical answer but output invalid step-by-step proofs.
- **Primary Bottleneck:** The bottleneck in LLM verification is symbolic proof construction, not semantic detection.

# Witness Validity Breakdown

| Model             | Precision (P) | Recall (R) | Validity (V) |
|-------------------|---------------|------------|--------------|
| GPT-5             | 40.1%         | 40.7%      | 41.4%        |
| Claude Sonnet-4.5 | 38.5%         | 39.9%      | 40.7%        |
| CWM               | 28.5%         | 25.9%      | 30.6%        |
| Qwen3-32B         | 22.6%         | 17.9%      | 24.8%        |
| GPT-4o            | 20.8%         | 22.7%      | 27.3%        |

- **Formatting Errors:** Extremely rare (0% in GPT models, < 5% in Claude/Qwen).
- **Content/Semantic Errors:** Main cause of failure; generated path graphs are syntactically valid JSON but fail SMT validation.

# Performance vs. Program Token Length

- Model reasoning score degrades consistently as input code length increases across token bins:
  - **Short (< 0.3K tokens):** High accuracy across all reasoning models (> 1.2 score).
  - **Medium (0.3K–6K tokens):** Performance drops moderately for proprietary models, sharply for open weights.
  - **Long (6K–84K tokens):** Severe degradation due to tracking multi-variable state spaces over extended contexts.
- Indicates a **long-horizon semantic reasoning bottleneck** in complex control flows.



# Failure Modes Across Subcategories

- **Control-Flow & Loops:** Near-zero failure rates on standard deterministic loops and recursion.
- **BitVectors & Heap:** Moderate accuracy; occasional errors when dealing with pointer aliasing or integer overflow bounds.
- **Concurrency Multi-Threading: Highest failure concentration.** Thread interleavings and non-deterministic scheduling present major challenges for LLM symbolic tracking.

# Divergence-Preconditions Formulation

- **Motivation:** GraphML witness automata are overly rigid and verbose for language models to generate cleanly.
- **Alternative Approach:** Prompt models to output a **logical precondition expression** over non-deterministic variables (e.g.,  $x < 0 \wedge y = 0$ ).
- **Theoretical Basis:** Represents the weakest divergence precondition (negation of weakest precondition under Hoare logic).
- **Validation Method:** Logical equivalence with ground truth verified automatically via the **Z3 Theorem Prover**.

# Domain Witness vs. Graph Witness Results

| Model             | Domain Witness (Logical) |             | SV-COMP Witness (Graph) |             |
|-------------------|--------------------------|-------------|-------------------------|-------------|
|                   | Pass@1                   | Pass@3      | Pass@1                  | Pass@3      |
| Claude Sonnet-4.5 | <b>0.87</b>              | <b>0.96</b> | <b>0.80</b>             | <b>0.83</b> |
| CWM               | 0.82                     | 0.96        | 0.75                    | 0.83        |
| Qwen3-32B         | 0.85                     | 0.94        | 0.77                    | 0.83        |

- Precondition expressions yield significantly higher accuracy (**Pass@1** and **Pass@3**).
- Significantly more token-efficient, human-interpretable, and easier to validate.

# Key Conclusions

- Frontier reasoning LLMs achieve empirical performance highly competitive with specialized formal verification tools on termination tasks.
- Pure classification accuracy is strong; the primary bottleneck lies in **formal symbolic proof generation**.
- Test-Time Scaling (TTS) via consensus voting substantially reduces false positives and improves overall evaluation scores.
- Program termination provides a benchmark for evaluating LLM limits on undecidable problems.

- **Neuro-Symbolic Systems:** Combine LLM semantic intuition with symbolic verifiers (e.g., SMT solvers) for hybrid proof generation.
- **Real-World Codebases:** Expand evaluation from self-contained benchmarks (SV-COMP) to repository-level continuous integration suites.
- **Alternative Proof Formats:** Adopt logical divergence preconditions in place of complex witness graph formats.
- **Broader Undecidable Tasks:** Extend methodologies to memory safety, reachability, and program equivalence.

# References



O. Sultan, J. Armengol-Estapé, P. Kesseli, J. Vanegue, D. Shahaf, Y. Adi, and P. O'Hearn.

*LLMs versus the Halting Problem: Characterizing Program Termination Reasoning.*

[arXiv:2601.18987](#), 2026.



D. Beyer and J. Strejček.

*Improvements in Software Verification and Witness Validation: SV-COMP 2025.*

[TACAS](#), pp. 151–186, 2025.



C. Metta et al.

*PROTON: Non-termination Analysis in C Programs.*  
[SV-COMP](#), 2024.

# Thank You!

Questions?

**MDL23CS001 23CSB01 AADITH N**  
CSQ413 Seminar